

Task 4: Exploitation & System Security

Theory Notes — Days 37–48

1. Penetration Testing Methodology

Penetration testing is a controlled, authorized simulation of a cyberattack on a system, network, or application to find security weaknesses before real attackers do. It always follows a structured process so that findings are reliable and reproducible.

The Five Phases

- **Reconnaissance:** Gathering information about the target (IP addresses, domains, employees, technologies used) through passive or active methods.
- **Scanning:** Actively probing the target to discover open ports, running services, and software versions (e.g., using Nmap).
- **Exploitation:** Using the information gathered to actually break into the system by exploiting a known vulnerability.
- **Post-Exploitation:** Once access is gained, determining what data or further access can be obtained, and how deep the compromise could go.
- **Reporting:** Documenting every step, evidence, and impact, along with recommendations to fix the vulnerabilities found.

Every step must be documented in detail — screenshots, commands used, and observations — because a pentest is only useful if its findings can be understood and acted upon by the client's security team.

2. Exploitation with Metasploit

Metasploit is an open-source penetration testing framework that contains a large database of known exploits for various operating systems, applications, and services. It allows testers to safely simulate real attacks in a controlled way.

Key Concepts

- **Exploit:** A piece of code that takes advantage of a specific vulnerability in software to gain unauthorized access or control.
- **Payload:** The code that runs on the target system after the exploit succeeds — for example, code that opens a shell.
- **Reverse Shell:** A connection where the compromised (target) machine connects back to the attacker's machine, giving the attacker command-line access to it. This is used because it often bypasses firewall rules that block incoming connections.
- **Meterpreter:** An advanced Metasploit payload that gives a rich, interactive shell with many built-in post-exploitation commands.

Post-Exploitation Commands

- **sysinfo:** Displays information about the compromised system (OS, architecture, hostname).
- **hashdump:** Extracts password hashes stored on the system, which can later be cracked offline.

Metasploitable2 is intentionally built with old, vulnerable services (like vsftpd 2.3.4) specifically so learners can practice exploitation legally and safely in an isolated lab.

3. Password Attacks

Password attacks aim to recover or guess valid credentials so an attacker can log into a system as a legitimate user.

Brute-Force Attacks (Hydra)

A brute-force attack systematically tries many username/password combinations against a login service (like SSH, FTP, or a web login) until one works. Hydra is a fast, popular tool for automating this against network login services. It can use a wordlist (a file of common passwords) to make the attack more efficient than trying every possible combination.

Password Cracking (John the Ripper)

Unlike brute-forcing a live login, password cracking works offline on password hashes that have already been obtained (e.g., via hashdump). John the Ripper takes these hashes and tries to reverse-engineer the original password by hashing guesses (from a dictionary or brute-force) and comparing them to the stolen hash. This is possible because many hashing algorithms, while one-way, are fast enough to guess against at scale unless properly salted and stretched.

Why This Matters

- Weak or reused passwords are one of the most common ways attackers gain initial access.
- Understanding these attacks helps in recommending defenses like account lockout policies, multi-factor authentication, and strong hashing algorithms (bcrypt, Argon2).

4. Social Engineering (Simulation Only)

Social engineering is the manipulation of people, rather than systems, to gain access to information or systems. It exploits human psychology — trust, urgency, fear, or curiosity — instead of technical flaws.

Phishing

Phishing is the most common form of social engineering: an attacker sends a fake but convincing message (often email) that tricks the victim into clicking a malicious link, entering credentials on a fake login page, or downloading malware.

Why Simulation Only

In this task, phishing is practiced strictly as an awareness exercise — creating a simulated phishing page to demonstrate how convincing these attacks can look, without ever targeting real people outside a consenting, controlled environment. Real-world phishing simulations are only conducted with explicit organizational authorization.

Awareness Training Focus

- Checking sender email addresses carefully, not just display names.
- Hovering over links before clicking to see the real destination URL.
- Being cautious of urgent or threatening language designed to rush decisions.
- Never entering credentials on a page reached via an email link — navigating directly to the site instead.

5. Malware Basics

Malware (malicious software) is any program designed to damage, disrupt, or gain unauthorized access to a system. Understanding how malware behaves is essential to detecting and defending against it.

Static vs Dynamic Analysis

- Static Analysis: Examining the malware's code or file structure without actually running it — for example, looking at strings, file headers, or disassembled code to understand what it might do.
- Dynamic Analysis: Actually running the malware in a controlled, isolated environment (a sandbox) and observing its real behavior — what files it creates, what network connections it makes, what registry changes it performs.

Sandboxing

A sandbox is an isolated virtual environment completely separated from real systems and networks. It allows a benign or suspicious sample to be executed safely, so its behavior can be studied without any risk of infecting the analyst's actual machine or network.

6. System Hardening

System hardening is the process of reducing a system's attack surface — the number of ways it could be exploited — by proactively securing it before an attack happens.

Key Practices

- Applying Security Patches: Regularly updating software and the OS closes known vulnerabilities that attackers could otherwise exploit.
- Firewall Configuration: Setting up rules to block malicious or unnecessary traffic, allowing only required ports and services to communicate.
- Disabling Unused Services: Every running service is a potential entry point; turning off services that aren't needed reduces the number of things that can go wrong.

Together, these practices follow the principle of minimizing the attack surface — the fewer open doors a system has, the fewer opportunities an attacker has to break in.

Summary

Task 4 covers the practical exploitation workflow used in real penetration tests — from following a structured methodology, to using Metasploit for exploitation, to attacking weak passwords, to understanding social engineering and malware, and finally hardening systems against these same threats. This mirrors the mindset of a professional penetration tester: think like an attacker in order to build better defenses.